



Original article

Optimizing data retrieval latency in IoT through information centric in-network caching

Muhammad Ali Naeem^a, Meng Yahui^a, Ahmad Abrar^{b,*}^a The School of Science, Guangdong University of Petrochemical Technology, Maoming 525000, China^b School of Computing, Universiti Utara Malaysia, 06010 Sintok, Kedah, Malaysia

ARTICLE INFO

Keywords:

Internet of things

Information centric networking

Popularity-aware multiclass content caching

ABSTRACT

The integration of Information Centric Networking (ICN) with the Internet of Things (IoT) will meet the expectations of end users by providing an admirable network system. ICN enhances the IoT by utilizing in-network caching regarding data dissemination with the help of various paths labeled by the name of the path by which the data is returned. Caching is a significant technique for improving content accessibility, reducing hops while transferring data, and finally shortening data access time, which in turn improves IoT networks. This study develops a novel caching strategy that caches content in suitable locations at highly requested nodes, thereby improving the information caching efficiency of ICN-based IoT systems. When comparing the proposed caching scheme with other caching methods, it pay attention to the data retrieval latency, cache hit ratio, and the average number of hops. These results consistently show that the proposed strategy enhances cache performance by a high margin. The future context of the utilization of the specified caching strategy will lie in the advancements of fog, edge, and ad hoc networks concerning the concept of IoT and new trends like 5 G and 6 G.

1. Introduction

RECENTLY, the Internet of Thing (IoT) has enabled compelling worldwide systems as well as end-to-end data transfer systems [1]. The most well-known examples of these systems requiring different forms of communication services are those related to packet delivery, content, and e-commerce. Furthermore, IoT makes all the devices connected to the Internet. The present communication architecture employs particular IP connections to forward and route data packets. However, to support data-centric applications such as corporate intranets, an end-to-end communication infrastructure of this nature is ineffective [2]. Furthermore, numerous challenges have negatively impacted the Internet of Things (IoT) [3], which is the most evident component of IoE due to the present Internet architecture based on IP [4]. Therefore, there is a need to provide an optimum architectural design towards the improvement of the quality of the communication channel.

Consequently, Information Centric Networking (ICN), a noteworthy architectural shift, was developed to address for upcoming Internet applications. Thus, in-network caching enhances the communication environment for IoT-based scenarios represented by ICN. It allows a

multipath data distribution by applying a named-based approach to data acquisition [5,6]. In-network caching is one of the most effective ways to optimize the functionality of IoT networks and minimize the amount of used bandwidth and time needed to obtain data as well as consumed energy. Due to the generic nature of the IoT as well as the enormous amount of content and correspondingly many different layouts of heterogeneous devices, applications have to be scalable. To overcome the challenges with respect to the large amount of data dissemination, the ICN caching which provides a novel conceptual solution settles in-network storage so that the repetition of data transmission does not occur [7,8]. This functionality enables IoT networks to store and obtain frequently used content from any network nodes enhancing the data irretrievability for the end-users who depend on IoT [9]. Also, through caching, using ICN reduces the amount of time that content is downloaded by the publisher and also preserves energy efficient networking for the internet of things. According to the analyzed characteristics of the IoT networks based on ICN, caching offers numerous benefits [10]. For instance, the distributed content is cached at corresponding intermediate locations and, thereby, making the correlated connected devices more private and scalable since it conceptualizes the end-user from

* Corresponding author.

E-mail addresses: malinaeem7@gmail.com (M.A. Naeem), mengyahui@gdupt.edu.cn (M. Yahui), abrar_ahmad@ahsgs.uum.edu.my, ahmadabrar2011@yahoo.com (A. Abrar).<https://doi.org/10.1016/j.aej.2024.10.067>

Received 6 April 2024; Received in revised form 6 October 2024; Accepted 14 October 2024

Available online 29 October 2024

1110-0168/© 2024 The Author(s). Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

the publisher [11].

Energy efficiency and resilience may be enhanced by adopting constraint-oriented devices. While caching has numerous benefits, it also presents complications and limits when creating caching algorithms for networks, such as the IoT. Caching may make the usefulness of a target redundant if users store duplicate material in several places. This may result in a substantial number of cache misses, leading to a drop in overall efficiency. This might significantly harm regular users' service quality. To design a caching strategy for an IoT scenario based on ICN, specific node and content attributes must be specified for caching. The node's attributes include its centrality, distance from the publisher, remaining memory, and battery level. The publisher's attributes include the freshness, timeliness, and popularity of the information. Based on these insights, a caching technique is devised for specified features. In this study, a dynamic popularity-aware caching method is developed. To execute caching operations, contents are partitioned into multiple classes as per the proposed caching scheme. This suggested caching scheme allows the network to enhance its caching performance by reducing the likelihood of redundant content being selected as popular. It also increases the quantity of diverse content that has the potential to gain popularity, thereby enhancing network performance, as measured by the cache hit ratio [12]. The key contributions of this research are given in the following:

- An adaptive content selection mechanism based on a dynamic threshold, which raises the threshold to increase the availability of content by prioritizing frequently downloaded items.
- A content selection mechanism that raises the frequency threshold in order to increase the availability of content by including less frequently downloaded items in the selection process, as opposed to exclusively choosing the most popular items.
- A content caching mechanism that increases caching performance by determining the most effective node to cache frequently downloaded content.
- A content caching mechanism is implemented to ensure that less frequently downloaded contents are cached at the appropriate node, thereby optimizing the utilization of the overall cache space.
- An energy-efficient usage mechanism is designed as part of this approach based on this mechanism, buffering operations are executed.

The subsequent sections constitute the structure of the remaining paper. In addition to presenting relevant studies along with their performance evaluations, Section II outlines the current challenges and rationales. III presents the problem statement. Section IV describes the Popularity-aware caching scheme that is proposed. The outcome of the evaluation and specified metrics are described in Section V. The outcomes and comparisons between the proposed and extant caching schemes are detailed in Section VI. In section VII, the conclusion and directions for future research are finally discussed.

2. Related work and caching issues

ICN is regarded as an emerging paradigm that enables cache-enabled network communication in order to improve the efficacy of the network as a whole. Caching frequently retrieved data, it reduces the distance between the source and subscribers. In support of the IoT network paradigm, it possesses benefits over the IP-based Internet architecture. It addresses the most critical challenges currently faced in IoT environments, including energy consumption, delay, stretch, and strike rate. As a result, in order to improve the caching efficacy of IoT environments with respect to the hit ratio, information retrieval latency, and hop counts [13], several caching strategies were devised. These metrics are regarded as the most significant for enhancing the efficacy of IoT-based networks. Nevertheless, the effectiveness of an IoT network utilizing ICN is contingent upon the caching strategy that is implemented. The chosen

approach determines which network node will cache the transmitted information items, and as a result, network efficiency is optimized. As a result, selecting a caching strategy in accordance with the network environment is the most crucial factor in enhancing the overall efficacy of the network. At the moment, the caching judgments performed at the suitable network node are regarded as the most critical component of an IoT network based on ICN. The reason for this is that if the caching decision is made in the incorrect location, it will negatively impact all caching capabilities [14].

Recent research has concentrated on caching in IoT networks based on ICN, which is crucial and highly demanded for the cost-effective transmission of data to end-users [15]. The ICN facilitates caching at intermediary network nodes for Internet of Things-based scenarios, thereby providing numerous advantages such as increased energy efficiency, decreased data delivery, and publisher burden. Additionally, it increases the durability and robustness in IoT-based networks [16]. Diverse caching schemes based on ICN have been devised to enhance service quality in IoT-related scenarios. Yan et al., [17] introduced a cluster-based information caching scheme as a means to enhance resource utilization efficiency. Caching is determined by the caching node and the requested content to decrease the time required to retrieve the data. Nour et al., [18] developed Near ICN Cache Placement (NCP), which selects the most cost-effective node to minimize expenses associated with content caching, content delivery from the source to the intermediate node, and content dissemination to end-users. Pfender et al., [19] conducted a comparison of caching strategies by considering hop reduction ratio and data retrieval delay. The examinations were conducted on an actual testbed. Meddeb et al., [20] have recently introduced Least Fresh First (LFF), a content substitution scheme that incorporates the cache's freshness requirement. It was intended to increase the vitality of the content while it was being replaced. It is evident from the recently published IoT caching schemes based on ICN that the focus of the research is on cost, energy consumption, content freshness, and content redundancy.

Zhang et al., [21] have recently developed a longevity-based Cooperative Caching (LCC) scheme with the objective of increasing the longevity of IoT contents and the rate at which users make requests, thereby improving the power efficiency of IoT-based networks. In accordance with the LCC, IoT-based devices may remain in dormant mode to increase energy consumption. However, these services are only provided in the cooperative caching region at a single hop, whereas multi-hop cooperation in the IoT region remains a significant challenge. As a result, it is imperative to arrange for the majority of nodes to be in a state of dormancy, which diminishes the amount of data accessible to end users. Lightweight caching based on approximate Betweenness Centrality (ABC) was proposed by Pfender et al., [22]. ABC caches content on a centralized node according to a heuristics-based network topology. Furthermore, communication complexity and information retrieval latency are diminished. Additionally, Amadeo et al., [23] propose Caching Fresh and Popular Contents (CFPC) as a means to improve the cache hit ratio and decrease the latency associated with data retrieval. For NDN-based IoT networks, it was suggested that the

caching performance be enhanced by concurrently evaluating content prominence and lifetime. A caching scheme known as Centrally Controlled Caching (CCC) was proposed by Asmat et al., [24] as a means to enhance the energy efficiency of Internet of Things (IoT) devices. A centralized entity was developed to regulate and monitor the transmission of content among various Autonomous Systems (ASs) and Internet of Things (IoT) end devices. Therefore, the energy consumption is decreased. Furthermore, Serhane et al., [25] developed Energy-aware Caching Placement (EaCP) to enhance energy efficiency in wireless IoT environments through the incorporation of ICN-based in-network caching. EaCP seeks to reduce energy usage throughout the content distribution and caching processes. Naeem et al., [27] introduced the Efficient Popularity-aware Probabilistic Caching (EPPC) technique in an effort to improve the efficacy of content caching. The selection of

content is determined by the frequency of requests, with prominent nodes storing the most frequently requested content. When the cache of the leading node is overloaded, the neighbor node caches the content. However, EPPC does not account for the timeliness of content, which could result in leading nodes being utilized unnecessarily by storing unpopular content for an extended period. Moreover, PoolCache, which Alahmri et al., [26] designed to enhance the redundancy performance of IoT networks, was also developed. This is achieved inside a given network by aggregating and managing the cache of each node so that there is no redundancy. Candidate Node Selection Cache Placement (CNCSP) is a strategy proposed by Dhawan et al., [28] that determines the most suitable node for content caching based on the prominence and freshness of the content. Furthermore, when performing a caching operation, the tradeoff between cache occupancy, node energy, and freshness was taken into account. Additionally, it incorporates bloom filters to facilitate rapid content retrieval. Nonetheless, the execution of numerous parameters to perform a single caching operation amplifies the communication overhead.

Energy Reward-based Caching (ERC) is an energy-efficient caching strategy that was proposed in a study by Dinh et al., [29]. Energy efficiency is regarded as the primary determinant in optimizing information transfer and caching power consumption. Additionally, it offers mechanisms for content replacement and positioning. Nevertheless, this caching strategy was solely concerned with energy usage. The fundamental information regarding related studies, including their evaluation metrics, is presented in Table 1. Meng et al., [4] introduced the Priority-based Content Popularity-Aware (PCPA) scheme as an improved mechanism for caching content. This scheme selects and caches content at an intermediate node according to a node interest threshold value and popularity and freshness values. Nevertheless, this results in a greater number of redundant data replications due to the decreased node threshold value. Consequently, the majority of the cache is occupied by less popular content. Therefore, a suitable caching strategy is necessary for the network, which can function in various ways including minimizing the latency of content downloads and enhancing caching gain [30].

However, caching has encountered significant challenges such as extended information retrieval latency and strain. A limited subset of the numerous requests for various types of content that the network receives

is selected for caching along the data routing path. The majority of the content that is cached is redundant. It could result in cache accommodation issues, in which redundant information remains stored in the content repository because a small number of users alter the distribution of the content by requesting similar material. The content store accumulates redundant content upon receiving multiple requests for it, assuming it is popular by that time. Moreover, a subset of users consistently submits queries for redundant content in an attempt to increase its popularity gradually. As a consequence, the redundant content continues to gain popularity and is incorporated into the network cache, resulting in a decrease in cache visits [31]. Consequently, the caching efficacy of an IoT network based on ICN is diminished. As a result, ICN-based IoT caching continues to encounter challenges associated with erroneous data selection and data caching at multiple undesirable network nodes that store similar data items. On the other hand, the Genetic Algorithm (GA) is also used in handling such problems [32–34].

These challenges serve as impetus for the development of a novel IoT caching strategy based on ICN in order to improve caching performance overall. Nevertheless, in IoT caching modules based on ICN, the incorporation of content selection based on dynamic popularity calculation and dynamic requesting frequency measurement into caching schemes is largely neglected. Several research methodologies were suggested that took into account the popularity of the content, including Auto-Regressive (AR) caching, which utilizes regression parameters to determine the popularity of the content [35]. To execute the content caching judgments, the researcher, on the other hand, concentrated on probabilistic-aware caching schemes, node-based caching, centrality-based caching, and freshness-based caching [36]. In the current investigation, a novel caching scheme is devised to address the issue of content popularity [37]. The subsequent sections describe in detail the caching strategy that is being proposed.

3. Proposed caching strategy

The growth in technology coupled with awareness of IoT gadgets enhances the production rate of data in networks, and therefore prompt management of data is very crucial for enhancing service delivery. The most promising candidate in the field of improving data delivery in the IoT is NDN, which targets a content-oriented model. However, several

Table 1
Related Works and Performance Analysis.

Year	Author & References	Simulation Tool	ICN-IoT	Content Freshness	Content Popularity	Hit ratio	Latency/Delay	Energy Saving	Hop-Count	Cache Utilization	Content Eviction
2017	Yan et al. [17]	NDN-Sim	✓	X	✓	✓	X	X	✓	X	✓
2018	Nour et al., [18]	Barabasi-Albert	✓	X	X	X	✓	X	✓	✓	X
2018	Pfender et al. [19]	CCN-lite	✓	✓	X	✓	✓	X	X	X	✓
2019	Meddeb et al. [20]	CCNSim	✓	✓	X	✓	✓	X	✓	X	✓
2019	Zhang et al., [21]	Custom built C++	✓	✓	✓	X	X	✓	✓	X	X
2019	Pfender et al., [22]	CCN-lite'	✓	X	X	✓	✓	X	✓	X	X
2020	Amadeo et al., [23]	NDN-Sim	✓	✓	✓	✓	✓	X	X	X	X
2020	Asmat et al. [24]	NDN-Sim	✓	X	X	✓	✓	✓	X	X	X
2021	Serhane et al., [25]	Barabasi network	✓	X	✓	✓	X	✓	X	✓	X
2021	Alahmri et al. [26]	CCN-Sim	✓	X	✓	✓	✓	X	✓	X	✓
2021	Naeem et al., [27]	ICARUS	✓	X	✓	✓	✓	X	✓	X	X
2022	Dhawan et al., [28]	NDN-Sim	✓	✓	✓	✓	X	X	✓	X	X
2022	Dinh et al., [29]	COOJA	✓	X	✓	✓	X	✓	✓	X	X
2023	Meng et al., [4]	NDN-Sim	✓	✓	✓	✓	✓	✓	✓	X	X

caching paradigms in conventional NDN have their drawbacks concerning latency and energy consumption, which are critical in IoT applications since most IoT devices are resource-constrained and geographically distributed. No doubt characterized by an increasing number of IoT devices, NDN-based IoT caching strategies for data are still critical. However, the conventional solution of centralized caching comes with some constraints and, thus, incurs inefficiency and high power consumption. These inefficiencies can worsen with a larger number of users, leading to poor performance, high latency, or both, and high energy consumption. The widespread use of IoT devices have led to a significant increase in the amount of data generated. Data retrieval is a time-sensitive process, and therefore any form of delay has drastic effects, especially in applications where response time is highly pivotal.

Since many IoT devices are portable and/or battery-powered, many of them may have small batteries. Excessive database calls and frequent data access can shorten their battery life and lead to more frequent replacements, hence the high maintenance costs. Energy efficiency is crucial for sustaining several forms of IoT deployments. A number of devices connected to the IoT mean that managing data is a much larger task when it comes to being intelligent about it. Scalability issues arise as user numbers rise. IoT networks are dynamic in nature; this implies that the data access patterns or the status of the IoT network are constantly changing. Thus, it is crucial to maintain balance or experience failure when it comes to caching techniques to get rid of such flitting results, as noted next. In these traditional approaches to cache implementation, the existence of many multiples of the same information affects the networks unwisely. These are, in general, the lower latency in the data transmission, better energy consumption, and optimum content caching that led to a better user experience. The concept of big data in smart cities requires the efficiency of real-time data acquisition, particularly among providers and consumers. Moreover, it is not simple to obtain high cache hit rates and, at the same time, maintain the absorbed energy, which is not very high. Due to caching that is always enhanced for performance, a greater measure of power is deployed than is required, thus making it a challenge for IoT networks to provide service and, consequently, cutting the useful lifecycle of IoT devices.

Solving these caching problems derived from the NDN-based IoT is still complex because new techniques that can self-optimize and self-adapt to the value changes of information, energy consumption, and high performance are required to meet the new requirements of today's IoT applications. Several of the issues with caching with the NDN-based IoT have to do with the temporal characteristics of data accesses, using minimal delay and power, managing congestion, and adapting to variations in the produced data. The solutions to these problems are the reason for enhancing the efficiency, expansiveness, and evolution of IoT networks. These problems are solved by the PMCC strategy, which enhances data caching, so that request data traffic on the network is decreased, and hence the number of data requests that have to be sent over the Internet is eliminated. In line with this, the availability of data at the intermediate nodes based on PMCC's dynamic popularity-aware caching mechanism decreases the time of retrieval and consequently improves the performance of these important applications. PMCC also has measures to track content popularity and network conditions so that it equally adjusts to the changes. It also guarantees that the caching strategy is stable and useful as the network changes over time. The PMCC dynamically deals with the content and strategically caches only those items that are most frequently accessed in a given timeframe, thereby ensuring that it does not unnecessarily replicate data that has already been cached. Although this approach helps achieve maximum cache optimization, it also ensures that content diversity is closer to the user, increasing the network's total performance.(Table 2)

4. Proposed caching strategy

The capability of pre-caching the content is regarded as the most effective method for efficiently delivering user-requested data at the

Table 2

Symbols and the corresponding description.

Symbols	Description
HDFC	Higher Downloading Frequency Class
LDFC	Lower Downloading Frequency Class
C	Set of all contents
I	Set of all Interests
$F(C)$	Content downloading frequency
$I(C)$	Collection of Interests for the downloaded file content
DC_r	Sum of all recently downloaded contents
I_r	Total inquiries were made to fetch all the contents
$P(c_i)$	Probability of a user's Interest i_i being for content c_i
$F_{threshold}$	Frequency threshold to identify HDFC and LDFC
$ C \mid I \mid$	Cardinalities of the sets
$Popularity(c_i)$	Popularity of content c_i
T_{cache}	Content Caching time
α	Constant between content caching time and energy level.
C_{cached}	Number of cached contents
β	Constant between caching time cached contents
E_{level}	Energy level of a node
$I(N)$	Node between server and user
$f(m)$	Function to measure number of incoming Interests
δ	Interest threshold value
ω	user preference equal to

network interfaces. Moreover, it mitigates the stretch-related and low-latency requirements of the end devices while enhancing the overall efficacy of the network. Thus, by employing dynamic content popularity-aware caching techniques in ICN-IoT caching, the aforementioned issues, such as the caching of redundant content, can be alleviated. Nevertheless, current caching methods utilize request frequency as a criterion for determining which content is most popular. Such content will be considered popular based on the number of requests from users it has garnered. Consider two contents, denoted A and B. During a specific period, content A has been queried extensively by users, while content B has been accessed by fewer individuals and has received fewer inquiries in comparison to content A. As a result, content A will be designated as popular and stored at the intermediary network locations in accordance with the popularity calculation based on frequency. Conversely, despite receiving 90 % more requests than content A, content B will continue to be despised and a significant number of additional inquiries for content B will have to be transmitted to the distant content provider. Consequently, caching efficacy will be diminished.

Similarly, the queries made by a limited number of users will be fulfilled using the frequently cached popular content. Recurring requests will result in an increase in cache entries for particular content that is intended for a reduced user base. In this study, a Popularity-aware Multiclass Content Caching (PMCC) scheme is presented as a solution to these challenges. This scheme functions as a dual-pronged mechanism, encompassing both data collection and content caching. In order to execute caching operations, the contents are categorized into multiple classes in accordance with the proposed caching scheme. Additionally, the suggested caching scheme allows the network to decrease cache utilization by preventing the prioritization of redundant content based on its popularity. Additionally, it optimizes the network performance by increasing the quantity of diverse content to be cached, reducing the selection of redundant content, and enhancing the hit rats and access latency.

4.1. Content selection

The proposed caching strategy classifies contents into two groups based on their downloading frequencies. The Higher Downloading Frequency Class (HDFC) comprises frequently requested content, while the Lower Downloading Frequency Class (LDFC) includes less frequently requested content. Let $C = \{c_1, c_2, c_3, \dots, c_n\}$ be the set of all contents, $I = \{i_1, i_2, i_3, \dots, i_n\}$ be the set of all Interests for content, and

$F(C)$ represent the downloading frequency and for each content the frequency is given by Eq. 1:

$$F(C) = \{f(c_1), f(c_2), f(c_3), \dots, f(c_n)\} \quad (1)$$

Pending Interest Table (PIT) records, which contain the Interests of each piece of content that cannot be located at intermediate nodes, are utilized to determine the frequency of downloads. Denoted as $I(C) = \{i(c_1), i(c_2), i(c_3), \dots, i(c_n)\}$ represents the collection of Interests that have been created for the downloaded file content and $DC_T = |C|$ represents the sum of all recently downloaded file contents, while the formula for calculating the sum of inquiries produced recently about each file content is calculated through the Eq. 2:

$$I_T = \sum_{c=1}^n i_i \quad (2)$$

where I_T represents the total number of inquiries recently were made to fetch all the contents. The probability of a user's Interest i_i being for content c_i can be defined by Eq. 3:

$$P(c_i) = \frac{f(c_i)}{I_T} \quad (3)$$

where $f(c_i)$ represents the Interest frequency for content c_i while I_T represents the total number of inquiries recently were made to fetch all the contents. The primary metric utilized to determine the popularity of individual contents, as per the PMCC, is the dynamic threshold value. Downloaded content is split into two fundamental divisions, HDfC and LDfC, according to the dynamic threshold. The dynamic threshold is determined using a mean value, which is computed by dividing the sum of all requests from prospective users' inquiries by the entire number of recently downloaded contents. To separate the contents into multiclass, they are sorted in ascending order by their frequency counts in a list following the calculation of the mean value. As a consequence, HDfC content is designated when the quantity of requests for a specific piece of content exceeds the mean value. On the contrary, content that demonstrates a lower number of downloads than the average value is classified as LDfC in accordance with the content-selecting mechanism of PMCC. As a result, each piece of content acquires a popularity value proportional to the frequency of its downloading count. In order to distinguish between the HDfC and LDfC classes according to the downloading frequency threshold, the mean value is calculated by the following Eq. 4:

$$F_{threshold} = \frac{I_T}{DC_T} \quad (4)$$

where $F_{threshold}$ shows the frequency, I represents the aggregate quantity of Interests generated for download the entire number of contents DC_T . Based on frequency threshold value, requested contents are divided into two classes: HDfC and LDfC, which denote the sets of contents in the higher and lower downloading frequency classes, respectively. Let ρ be the proportion of contents that are in the HDfC, ρ can be calculated through the Eq. 5:

$$\rho = \frac{|HDfC|}{|C|} \quad (5)$$

where $|HDfC|$ and $|C|$ are the cardinalities of the sets HDfC and C respectively. The content selection mechanism chooses a content to cache based on its class and popularity within that class. The set of all contents for them the Interests were generated is sorted in ascending order regarding their Interest frequency counts, denoted as: $c_1, c_2, \dots, c_m$. Therefore, if a content c_i has a higher frequency count $f(c_i) > T$, c_i is included HDfC class which can be defined by the following Eq. 6:

$$HDfC = \{c_i : f(c_i) > T\} \quad (6)$$

where $c_i \in C$. On the other hand, if a content c_j shows frequency threshold less than or equal to $f(c_i) \leq T$ then content c_j is included in

LDfC class and it is given by 7:

$$LDfC = \{c_i : f(c_i) \leq T\} \quad (7)$$

where $c_j \in C$. As a result, all contents receive some popularity value regarding their downloading frequencies, which can be defined by Eq. 8:

Algorithm 1. : Find the HDfC and LDfC.

Input:

C: Set of all contents

$f(c)$: Downloading Frequency for a content

$F_{threshold}$: Frequency threshold

I_T : Total number of Interests for all contents

DC_T : Total number of downloaded contents

Output:

c_i = Selected Content

Step 1: Calculate the frequency threshold $T = F_{threshold}$

$$a. \quad F_{threshold} = \frac{I_T}{DC_T}$$

Step 2: Divide contents into HDfC and LDfC

$$a. \quad HDfC = \{c_i : f(c_i) > T\}$$

$$b. \quad LDfC = \{c_i : f(c_i) \leq T\}$$

Step 3: Calculate the popularity value for each content

a. **for** each c_i in C **do**

$$b. \quad Popularity(c_i) = \frac{f(c_i)}{I_T}$$

Step 4: Select the content to cache based on its class and popularity.

a. $Num_{rand} = randomUniform(0, 1)$ // Generate a random number between 0 to 1

b. **if** $Num_{rand} < \rho$ **then**

c. $c_i = selectContentHDfC(HDfC, F, popularity)$

d. **else**

e. $c_i = selectContentLDfC(LDfC, F)$

f. **return** c_i

Step 5: Choose content from HDfC based on Popularity value

Procedure selectHDfC(HDfC, F, popularity)

a. $popularity_{Total} = 0$

b. **for** c in HDfC **do**

c. $popularity_{Total} += popularity(c)$

d. $Num_{rand} = randomUniform(0, 1)$

e. $cumulative_{prob} = 0$

f. **for** c in HDfC **do**

g. $cumulative_{prob} += popularity(c)/popularity_{Total}$

h. **if** $Num_{rand} \leq cumulative_{prob}$ **then**

i. **return** c

j. **return** HDfC[last_index] // When loop did not return any value

Step 6: Choose content LDfC uniformly at random

Procedure: selectLDfC(LDfC, F)

a. $rand_{index} = randomInteger(0, len(LDfC) - 1)$

b. **return** LDfC[random_index]

$$Popularity(c_i) = \frac{f(c_i)}{I} \quad (8)$$

where I denoted the number of inquiries made for content c_i while $f(c_i)$

shows the Interests frequency. Let c_i denote the content to cache and if c_i content is selected from HDFC, it is chosen with probability proportional to its downloading frequency within HDFC and given by Eq. 9:

$$P(c_i \in \text{HDFC}) \propto f(c_i) \quad (9)$$

where $c_i \in C$. On the other hand, if a content c_j is selected from LDFC, it is chosen uniformly at random from LDFC and probability is given by Eq. 10:

$$P(c_j \in \text{LDFC}) = \frac{1}{|\text{LDFC}|} \quad (10)$$

where $c_i \in C$. By employing a frequency threshold and a dynamic popularity-aware content selection mechanism, this system guarantees that content that is frequently requested is cached at the network nodes more frequently while permitting content that is less frequently requested to remain cached. The Algorithm 1 first computes the downloading frequency $f(c)$ for each content c by counting the number of Interest in I . It then uses this to compute the probability of Interest being for each content and divides the contents into HDFC and LDFC based on their downloading frequencies. The algorithm then repeatedly selects contents to cache until it has selected demanded contents. For each selection, it first selects a content from HDFC with probability p , based on its downloading frequency within HDFC. If a content is selected from HDFC, it is chosen with probability proportional to its downloading frequency within HDFC. Otherwise, it selects a content uniformly at random from LDFC.

4.2. Content caching

The initial stage of this caching strategy is called "content selection," which is determined by the frequency and downloading history of the content. The content caching method commences in the second phase, subsequent to the conclusion of the content selection phase. During this stage, the cache or prioritization of the most and least downloaded content is executed as the requested content is transmitted to the end devices which initiated the requests to retrieve the content. To efficiently cache content at intermediate nodes, several parameters must be fulfilled, including the number of nodes along the downloading path, the total path distance, the content caching node, the available cache capacity, and the transmitted content. The content caching procedure can be accomplished through the utilization of various concepts, including probability-caching, centrality-caching, node-caching, and popularity-caching. In contrast, PMCC caches the content based on the node that exhibits the same Interest in the content. The determination of the content caching position (caching node) involves measuring and calculating the number of incoming Interests and predictor history. Subsequently, items are cached at the effective nodes. To ascertain the caching node, PMCC evaluates a variety of factors, including the relationship between cache energy and their respective capacities.

4.3. Relation between energy and cache capacity

The calculation of the content caching time is a critical element in improving the caching efficacy of NDN networks based on IoT. The limitation of the available cache space pertains to the contents that are transmitted. Consequently, reserving all the distributed content at the intermediary nodes is unattainable. Notwithstanding this, caching efficacy can be enhanced through the implementation of an ideal cache management strategy. Certain strategies assign static time to individual nodes, which specify the amount of time that content can be stored at that node. While certain approaches dynamically address this issue, such as the probabilistic caching strategy [38], which handles the content caching time dynamically, the distance from the end-users increases the content caching time. Allocating a particular duration for a cached content is thus the most promising technique to enhance the caching performance. The PMCC algorithm utilizes the energy level of a network node to decide the duration of a cached content. The temporal duration

for which the content is cached at a given node is determined by its energy level. Specifically, content will be cached for a limited duration if the node has a low energy level. However, the content can remain at a node for an extended period as the energy level increases.

Algorithm 2. Content Caching Duration.

Input:

E_{level} of a network node

C_{cached} cached content at a node

Output:

T_{cache} content caching time

Constants:

α : show the relationship between content caching time and energy level.

β : Proportionality constant to identify the rate of change between content caching time and the number of cached contents.

Procedure: *CachingOperation*(E_{level} , T_{cache} , *threshold*)

1. $T_{cache} = \alpha(E_{level})$
2. **if** $E_{level} > \text{threshold}$ **then**
3. **return** Content Caching at current node
4. **else:**
5. **return** Caching content at next node w
6. $T_{cache} = \frac{\beta}{C_{cached}}$
7. **return** final T_{cache}

In PMCC, the energy level of a node is checked to perform a content caching operation. If the energy level of a node is higher than 10 percent, the caching operation is performed. Otherwise, the content is cached at the next node in case of a low energy level. The reason is that, if the most popular content is cached at the node having lower energy level, it may happen that the usage of node will be increased because a large number of Interests will be reached at the that node and node has to respond all the Interests. In this way, the energy level of the node quickly be lower down and cannot perform the demanded executions by sending the demanded content to all users. Moreover, caching duration in PMCC is dynamically changed with increasing or decreasing the number of accommodated contents. If the number of contents is higher the content caching duration will be decreased while the content will be cached longer with the smaller number of contents. In this way, content accommodation gets enhanced and maximum content can be accommodated at the intermediate node, increasing the caching gain and reducing response latency. Let E_{level} be the energy level of an NDN-based IoT caching node while T_{cache} shows the data caching time and C_{cached} be the number of cached contents at a network node then to measure the content caching time regarding the energy level of a caching node can be defined using energy level with content caching time as given by the Eq. 11:

$$T_{cache} = \alpha(E_{level}) \quad (11)$$

where T_{cache} is the content caching duration for a content which is calculated in seconds while E_{level} represents power of the battery capacity of remaining charging and α is the constant to shows the relationship between content caching time and energy level. Therefore, the higher the energy level longer the content will be cached at a node. In PMCC the content caching time dynamically is adjusted based on the number of cached contents. If the number of cached contents is greater the caching duration will be reduced. On the other hand, the contents have longer cache duration with the smaller number of cached contents. In this way, a large number of diverse contents can be accommodated within the network cached. Therefore, the content caching time is

inversely proportional to the number of contents, and it can be defined by Eqs. 12 and 13:

$$T_{cache} \propto \frac{1}{C_{cached}} \quad (12)$$

$$T_{cache} = \frac{\beta}{C_{cached}} \quad (13)$$

where β is the proportionally constant that identifies the rate of change between content caching time T_{cache} and the number of cached contents C_{cached} . Therefore, the content is cached based on the energy level, and the energy level of a node should be greater than 10 percent of total. The threshold value can be defined by the following Eqs. 14 and 15:

$$E_{level} > 0.1 \quad (14)$$

$$E_{level} < 0.1 \quad (15)$$

If a node showing energy level lower than 10 percent of the total energy the content will be cached on the next node according to the Eq. 15. Algorithm 2 shows the relation between node energy level and cache capacity.

Algorithm 3. Identify the effective caching nodes

Input:

Number of Nodes: N
 Number of Interests: I
 Number of Contents: C
 Provider Node: $P(n)$

Output:

Content Caching Decision

Step 1: Number of Interest at each node

Procedure: Count NewInterests (node, Interests)

- a. count = 0;
- b. **for** each Interest in Interests **do**
- c. **if** node in $I(N)$ of Interest **then**
- d. count = count + 1;
- e. **return** count

Step 2: The Interest threshold at each node

Procedure: CachingNode ($N, C, I, E, P(n)$)

- a. $\delta = \frac{|I|}{\omega}$
- b. Decision = empty dic
- Step 3:** Calculate $f(m)$ for each node m in N for each m in N
- a. $f(m) = \text{count NewInterests}(m, I)$

Step 2: Identify the effective nodes E

- a. $E = \text{empty set}$
- b. **for** each m in N **do**
- c. **if** $f(m) > 1$ **then**
- d. $E.add(m)$

Step 3: Determine caching decisions

- a. **for** each node in N **do**
- b. **if** node in E and $f(m) > \delta$ **then**
- c. decision[node] = 1 // Cache content
- d. **else:**
- e. decision = [node] = 0 // Content cannot be cached here
- f. **return** decision

4.3.1. Effective caching nodes

In order to determine the effective intermediate nodes for content caching, suppose several Interests from different users are generated and sent to download a particular content from the provider node. The requests travel from the different network nodes and reach provider node. Therefore, all the nodes that were used to transmit the users' requests create an entry for each request. As the requests were sent from the different users toward the provider node, some nodes will be common and used to forward these incoming Interests to the sources. As a result, the common nodes are selected regarding the number of requests and are called effective nodes. Therefore, if the numbers of pending entries of requests are equal or greater than 80 percent of the total amount of Interests to the number of received Interests at the provider node the node will be selected to be cached the requested content. In this way, the redundant content's replications are minimized, and it helps to accommodate diverse content. In some scenarios, if all the common nodes are selected for content caching, the PMCC strategy will perform similar caching operations and behave like CEE or ProbCache which increases the data redundancy. Therefore, to improve the cache accommodation and heterogeneous caching operations, the PMCC selects one-third numbers of requests at each node.

Therefore, in order to determine the effective caching nodes let $N = \{n_1, n_2, \dots, n_n\}$ represents all the nodes in a network and $I = \{i_1, i_2, \dots, i_n\}$ be the total number of Interests generated from the multiple users to fetch the content c_i from the set of content $C = \{c_1, c_2, \dots, c_n\}$ and initially c_i stored at provider node $P(n)$. Each user Interest is linked with source node $i(s)$ and destination node $i(d)$. Consider $E = \{e_1, e_2, \dots, e_n\}$ be the effective caching nodes that are chosen for the content caching. Each node creates an entry as the Interest i_i passes through source $i(s)$ to the destination node $i(d)$. Therefore, the total nodes between server and user can be defined as $I(N)$ and it is given by the following Eq. 16:

$$I(N) = \{i_1(n_1), i_2(n_2), \dots, i_n(n_n)\} \quad (16)$$

The nodes that are involved to transmit the Interest i_i between source node $i(s)$ and destination node $i(d)$ and when the source node is not as the sever or it can be any of the intermediate node. Therefore, when the Interest i_i pass through several node and satisfied at the i th node, the number of nodes can be defined by Eq. 17:

$$I(N) = \{i_1(n_1), i_2(n_2), \dots, i_i(n_i)\} \quad (17)$$

The effective are the nodes which are used to transmit the users incoming Interests I to the provider node $P(n)$. Let $f(m)$ is the function which is used to measure number of incoming Interests for the intermediate network nodes. These nodes can be calculated through the following Eqs. 18 and 19.

$$f(m) = \{i \in I | m \in I(N)\} \quad (18)$$

The number of effective nodes E can be defined as:

$$E = \{m | m \in N, f(m) > 1\} \quad (19)$$

In order to identify whether the content c_i can be cached at node n_i or not it defines a caching decision function that shows two outputs in binary (1,0) form and therefore, the caching decision will be executed based on the following conditions 20:

$$\text{Caching Decision} = \begin{cases} 1, & m \in E \text{ and } f(m) > \delta \\ 0, & \text{otherwise} \end{cases} \quad (20)$$

where $\delta = \frac{|I|}{\omega}$, if node n_i along the data downloading path belongs to the set of E and the effective nodes has $|E|$ shows greater user preference equal to $\omega = 80$ percent of the user preference at $P(n)$ node, then the output will be 1 and the node will be chosen as caching node and the selected popular content will be cached at that node. Otherwise, it shows

0 output value and the content will forward to the next node. **Algorithm 3** identifies the content caching nodes based on the Interest threshold value $\delta = \frac{|I|}{\omega}$ along the data deliver path.

Algorithm 4. : Caching HDFC/LDFC.

```

Input:
    Effective nodes or edge nodes

Output:
    Cache the incoming content at effective node or edge node

#Step 1: Find the node type is effective or edge node
Procedure: cacheContent (node,nodeTypes)
a.   cacheContent = []
b.   for i = 0 to length(nodes) - 1 do
c.       node = node[i]
d.       nodeType = nodeTypes[i]
e.       if nodeType == effective then
f.           cachedContent[node] = cacheHDFC(node)
g.       elseif nodeType == edge then
h.           cachedContent[node] = cacheLDFC(node)
i.   return cachedContent

#Step 2: Cache the HDFC
Procedure: cacheHDFC(node)
a.   if HDFC available (node) then
b.       addToCache(node, HDFC)
c.       return cache HDFC
d.   else
e.       return HDFC not available

#Step 3: Cache the LDFC
Procedure: cacheLDFC(node)
a.   if LDFC available (node) then
b.       addToCache(node, LDFC)
c.       return cache LDFC
d.   else
e.       return LDFC not available

```

Algorithm 4 represents the caching of HDFC and LDFC at effective nodes and edge nodes respectively. Fig. 1 illustrates the content selection and content caching mechanisms of the PMCC caching strategy. Interests were sent from a set of IoT-based devices such as $Devices = \{d_1, d_2, \dots, d_n\}$ where $n = 7$. The total number of Interests $I_T = \{i_1, i_2, \dots, i_n\}$ where $n = 10$ to download the total number of contents $DC_T = \{x_i, x_j\}$ as presented in given scenario. Initially, the contents x_i and x_j are published by the $PublisherP_1$ at Provider node v_6 . At the same time when the

Interests are reached at the node v_6 , the Interest frequency $f(x_i)$ and $f(x_j)$ is calculated to measure the dynamic frequency threshold value $F_{threshold} = \frac{I_T}{DC_T} = 5$. Based on the frequency threshold the $(x_i \in HDFC)$ is considered as higher popular content and therefore (x_i) is suggested to be stored at effective network nodes. On the other hand, the content $(x_j \in LDFC)$ is considered as lower popular content and suggested to be stored at only at edge node v_1 . Therefore, the energy condition $E_{level} > 0.1$ is checked and according to the energy level the content x_j is cache at node v_1 as shown in given scenario (Fig. 1). The content x_j is cached only at edge node v_1 because the content x_j is fetched only from the one device d_2 which is directly connected to the edge node v_1 . However, for the caching of x_i the effective nodes are determined and in the current

scenario the node v_4 and v_5 are selected as the effective nodes based on the shortest distance from the data demanding devices $(d_1, d_3, d_4, d_5, d_6, d_7)$ and Interests preference threshold $\delta = \frac{|I|}{\omega}$. Consequently, when a node exhibits a greater than $\omega = 80\%$ Interest for content, that node will cache the content. However, the node v_5 cannot fulfil the energy requirements ($E_{level} < 0.1$) and thus the content x_i is cached at the next node v_4 as shown in the given scenario.

5. Evaluation

To assess the effectiveness of PMCC, the NDN-SIM [13] simulator, which is a subset of the NS3 simulator, is utilized to benchmark and simulate the proposed ICN-based IoT caching strategies. To replicate the present situation, a rectangular area measuring $250\text{ m} \times 250\text{ m}$ is chosen, and 80 terminals are arbitrarily distributed across it. Four nodes alongside the grid topology edges are selected as providers (content producers) and the 45 nodes among the edge nodes are selected as sink nodes and consumer nodes are chosen as 30. All the nodes having IEEE 802.11a wireless technology connections to transmit the data item. Each content or data chunk has 512 bytes in size each node's cache size is selected as 10 chunks in the first scenario and 100 data items for the second scenario or each node can cache 10 data chunks in the first scenario 100 data chunks in second scenarios. The Zipf popularity distribution is chosen with alpha values range from 0.2 to 1.2. Table 3 shows the simulation parameters and their corresponding values.

5.1. Comparing strategies

To assess the efficacy of the proposed caching strategy, its performance is evaluated in comparison to six other caching strategies: Popularity-aware Closeness Centrality Caching (CaCC), ProbCache (P (0.5)), Most Popular Cache (MPC), Caching Fresh and Popular Content (CFPC), and Caching Fresh and Popular Content (CFPC). The following strategies are delineated:

5.2. Cache everything everywhere

Cache Everything Everywhere (CEE) [39] provides a straightforward architecture for ensuring high data accessibility and availability. Conversely, CEE exacerbates the duplication of superfluous content, resulting in a decline in the overall performance of the network. Furthermore, by caching content across all network endpoints, it increases the utilization of the network cache. Additionally, it causes unnecessary cache utilization by caching objectionable content at every node. Consequently, it utilizes superfluous network resources.

5.3. ProbCache

The ProbCache (P(5.0)) was developed to share the fair allocation among the caching contents using multiple factors for each request to calculate the probability value [38]. Therefore, probabilistic values need to be calculated through various mathematical functions, such as time since birth and time since inception, to evaluate the probabilistic value for each data item, which increases the communication overhead. It needs additional cost to execute such parameters and reduces the overall performance. Moreover, it forwards all the incoming data items, considering their popularities, and therefore unwanted similar caching operations are performed at several places, which increases the eviction ratio. As a result, a number of requests are needed to traverse several hops to find the evicted popular data items from the publisher, increasing the average hop count and response latency [40,41].

5.4. Most popular cache

The Most Popular Cache (MPC) [37] employs a static threshold to

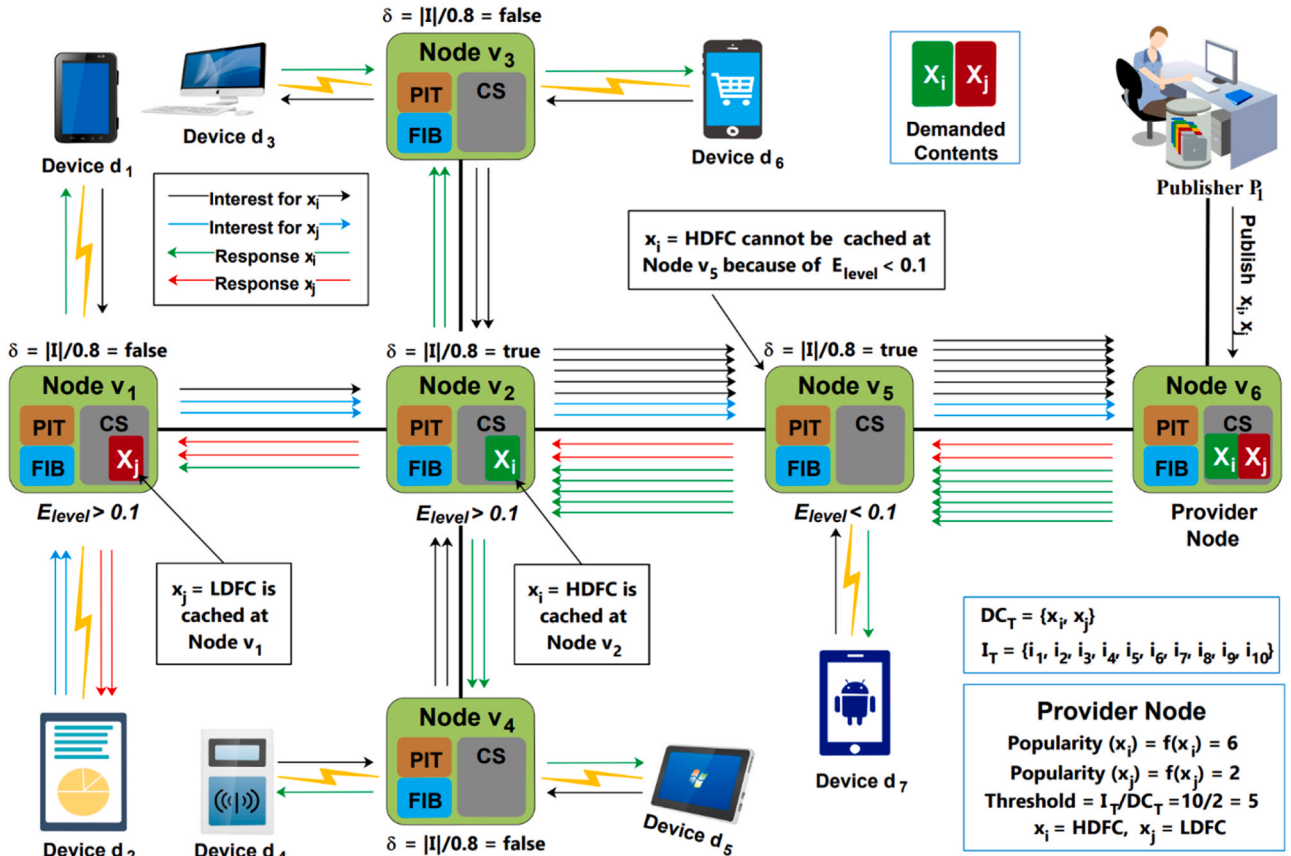


Fig. 1. Popularity-aware Multiclass Content Caching.

determine the popularity value of the selected content. The chosen contents are then cached at multiple nodes, as well as at neighboring nodes. Nevertheless, content is cached at intermediary network nodes irrespective of the user's level of Interest, thereby increasing the likelihood that the majority of the cache is used by content with a lower popularity tally. Consequently, the majority of user interests must be transmitted to the primary data providers, which increases the average journey count value. Furthermore, this results in an increase in redundant caching replications across multiple nodes, thereby reducing the likelihood of accommodating more popular content. This factor is critical in enhancing the overall performance of caching.

5.5. Caching fresh and popular contents

The caching decision in Caching Fresh and Popular Contents (CFPC) [23] is independently determined by each node, taking into consideration two parameters: the life of the content and its popularity tally. The prevalence of a piece of content is determined by summing the total number of requests from users for that content. Nevertheless, the lifetime is determined by the novelty value of the content. When a request is generated, the same process must be repeated at each caching node to accommodate the newly added content as the cache is populated. The majority of requests are therefore satisfied by the publisher.

5.6. Centrally controlled caching

Centrally Controlled Caching (CCC) [24] proposed to implement the Autonomous Systems (AS) based caching strategy, where a central entity Network Controlled Controller (NCC) manages the energy consumption within an AS. It also manages the data retrieval process between AS and end-users. Content Forwarding Table (CFT) and Network Content Table (NCT) are also used to complete the communication process

Table 3

Simulation Parameters and Corresponding Values.

Parameters	Description
Size of one chunk	512 bytes
Wireless technology	IEEE 802.11a
Cache-Size	10, 100 Data chunks
Mobility Model	Random walk mobility
Number of nodes	80
Number of user node	30
Network caching node	45
Number of content providers	5
Content popularity model	Zipf Distribution
Alpha values	0.2, 0.4, 0.6, 0.8, 1.0, 1.2
Topology size	(250 × 250) m
Radio coverage area	50 m
Simulation time	1800s
Refresh period	10 s, 20 s, and 40 s

between users and AS. However, several tables are used to complete one transmission, and all these tables need to be executed while receiving an Interest or content, increasing the communication overhead. Moreover, it does not provide any cooperative process to provide information about one AS to another AS. Therefore, most of the users' demanded contents need to be fetched from distant data providers, increasing the data retrieval latency and hop count.

5.7. Popularity-aware closeness caching

Popularity-aware Closeness Caching (PaCC) [42] was recently proposed to improve content caching using proximity and content popularity. In PaCC, the user interest rates are periodically updated to determine the content popularity. It implements a metric named

Popularity Aware Closeness (PAC) to calculate the caching node, where contents are cached at the closeness centrality nodes. It caches the content based on PAC value. Nevertheless, the number of contents is much greater than the available cache storage. Therefore, not all incoming contents can be cached at one node. As a result, content eviction increases, and most of the users' interests are forwarded to the server to download the demanded contents. This reduction in caching performance is evident in terms of average hop count, latency, and hit rate.

5.8. Performance metrics

The purpose of the experiments was to assess the effectiveness of the proposed PMCC and compare its characteristics with CEE, ProbCache (P (0.5)), MPC, CFPC, CCC, and CaCC. This comparison was made using various metrics, including the Cache Hit Ratio (CHR), Stretch Ratio (SR), and Data Retrieval Latency (DRL).

5.9. Cache hit ratio

CHR is calculated by dividing the sum of all memory accesses by the total number of hits. Its definition is the sum of all interests that satisfy the user's requirements from intermediate network locations. It can be measured by the Eq. 21:

$$CHR = \frac{\sum_{i=1}^n Cache_{hit}}{\sum_{T=1}^n Cache_{access}} \quad (21)$$

where $\sum_{i=1}^n Cache_{hit}$ represents the number of cache hits occurs and $\sum_{T=1}^n Cache_{access}$ shows the total number of cache access.

5.10. Data retrieval latency

Data Retrieval Latency (DRL) is referred to the life cycle (time) or duration to complete transmission of the user's Interests to the data provider and data to the user. In ICN-IoT scenarios, the data retrieval latency is measured in seconds or milliseconds, and it can be calculated using the following Eq. 22:

$$DRL = Time_{Interest} + Time_{data} \quad (22)$$

where $Time_{request}$ represents the time is taken by the Interests to reach at the caching node while $Time_{data}$ is the time taken by demanded content to the requested users.

5.11. Stretch ratio

The Stretch Ratio (SR) has referred to the hops traveled by the requests to fulfill the end user's demands and it can be measured by 23:

$$SR = \sum_j^N \frac{hop_i}{N(Hop_i)} \quad (23)$$

where hop_i represents the number of hops that the user's Interests completed to locate the intended content from the intermediary node, and N denotes the overall count of Interests generated. The steps traversed by the request from the inquiring node to the provider node are denoted by i .

6. Results and discussion

This section presents the comparisons and simulation results of the proposed and comparing strategies. This study evaluates the impact of Zipf popularity distribution and varying cache sizes on different ICN-based IoT caching strategies. Two different scenarios were established

in which one scenario shows the varying α parameters such as 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, while the cache sizes are taken as constant with two different sizes as 10 data chunks and 100 data chunks. However, in the second simulation scenario, the cache size is taken as varying from 10 chunks to 100 chunks, while α parameters are taken as constant and selected as the smallest value of 2.0 and largest value as 1.2. Therefore, different parameters and cache sizes were selected to check the correctness of the simulation results of comparing strategies. These scenarios are thoroughly explained in the following sections:

Multiple caching strategies such as PMCC, CFPC, P (0.5), CaCC, CCC, MPC, and CEE are compared in a common simulation platform in terms of the CHR, DRL, and SR. From the given result in Fig. 2(a, b), it is clear that the proposed PMCC performed better as compared to the benchmark strategies (PMCC, CFPC, P (0.5), CaCC, CCC, MPC, and CEE). The performance is increasing as the popularity value increases. However, the PMCC has outperformed all popularity values. The reason is that PMCC caches the highly downloaded data items near the end-users and therefore, most of the user's requests are satisfied from the closest nodes. Thus, the caching hit ratio becomes increases. The CEE caches all contents at all the available nodes and when the new content needed to be cached, several evictions are performed to accommodate the new data items. Moreover, in ProbCache (P (0.5)), the content is cached at multiple unwanted locations. Consequently, if the content is needed far from the provider node and the cache of all the nodes near the user is already filled thus, the content will be cached far from the users.

As compared to the CEE and ProbCache (P (0.5)), the MPC and CFPC performs better because these strategies cache the popular contents near the end-users. Consequently, multiple Interests are accomplished there from the most popular contents. According to the DRL (c, d) and the SR (e, f) perspectives, all strategies show performance similarly and with different popularity values. However, with all different popularity values, the PMCC outperformed due to its heterogeneous ability to cache HDLC and LDLC at appropriate locations (nodes).

As a result, the time required by PMCC to download the requested data items by users is reduced, and user requests need to make fewer hops to retrieve the contents from the nearby storage. Conversely, CEE and ProbCache exhibit inferior performance in terms of stretch and latency when compared to CFPC. Both strategies increase data redundancy by caching several copies of the same content at multiple nodes; to accommodate the new data contents, a significant number of eviction operations are executed. As a result, the majority of user queries must make multiple hops to obtain the necessary data items. As a result, the average number of steps is optimized, leading to an increase in the latency of data retrieval. Furthermore, the performance of MPC and CFPC has been superior to that of CEE and ProbCache owing to their capability of caching popular content. As a result, time and distance are diminished through the caching of data at the nodes.

MPC exhibits superior performance compared to CEE and ProbCache; however, it has attained inferior outcomes about of CHR, DRL, and SR. This is due to its implementation of a static criterion value (threshold = 2) and its practice of caching the chosen contents at multiple nodes, as well as at the nodes of its neighbors. Nevertheless, content is cached at intermediary network nodes irrespective of the user's level of interest, thereby increasing the likelihood that the majority of the cache is utilized by content with a lower popularity tally. Consequently, the majority of user interests must be transmitted to the primary data providers, which increases the average journey count value. MPC Furthermore, these results in an increase in redundant caching replications across

multiple nodes, thereby reducing the likelihood of accommodating more popular content. This factor is critical in enhancing the overall performance of caching. The calculation of content prominence in CFPC involves the aggregation of all user queries received for a given piece of content. Nevertheless, the lifetime is determined by the novelty value of the content. The process of accommodating newly added content at each caching node

is iterative as the cache is populated each time an inquiry is generated. As a consequence, the majority of requests are satisfied by the publisher, leading to a decrease in the overall caching efficacy.

CCC introduces several tables that are used to complete one

transmission, and all these tables need to be executed while receiving an Interest or content, increasing the communication overhead. Moreover, it does not provide any cooperative process to provide information about one AS to another AS. Therefore, most of the users' demanded

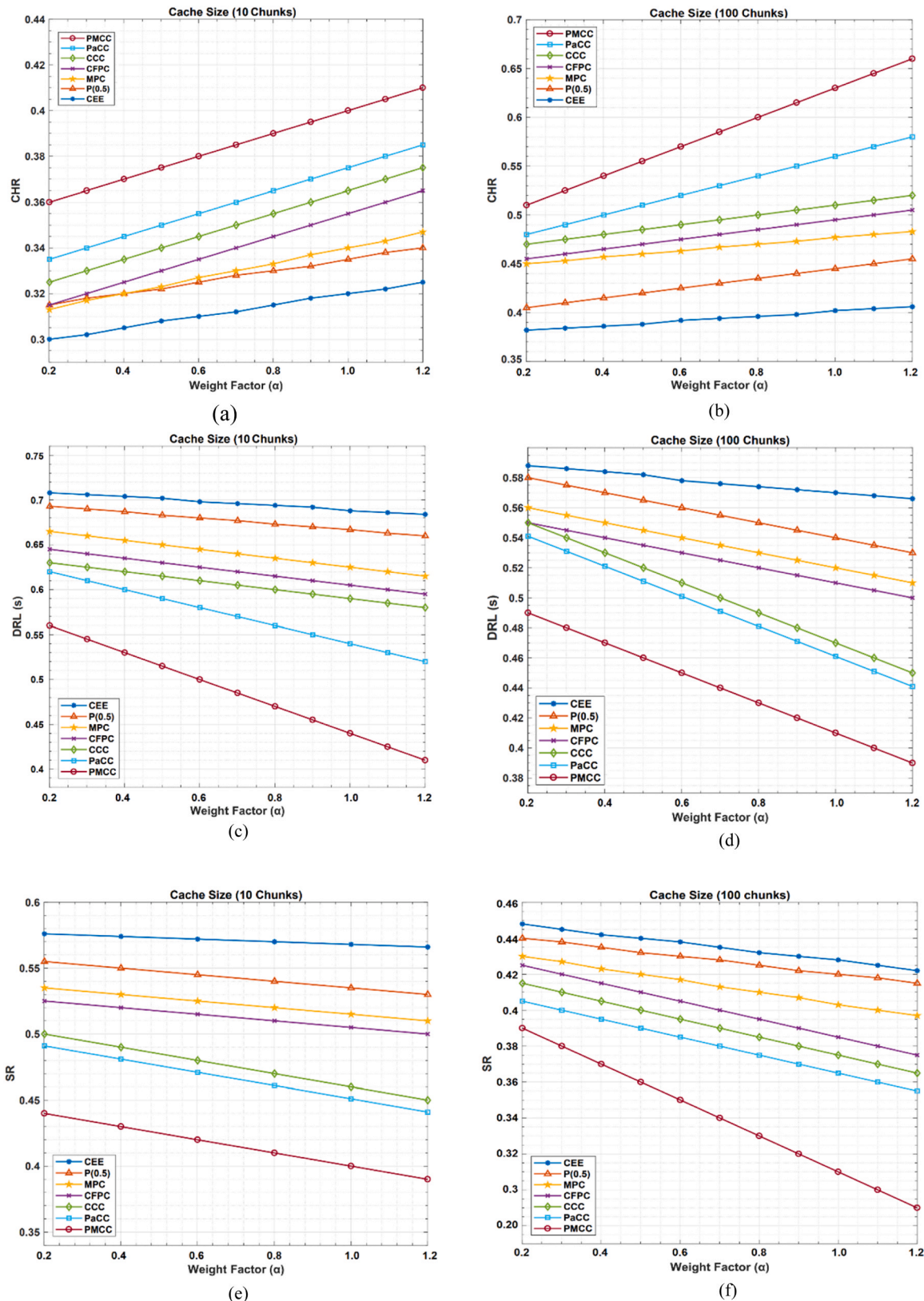


Fig. 2. Simulation results on CHR, DRL, and SR with Content Cache Size.

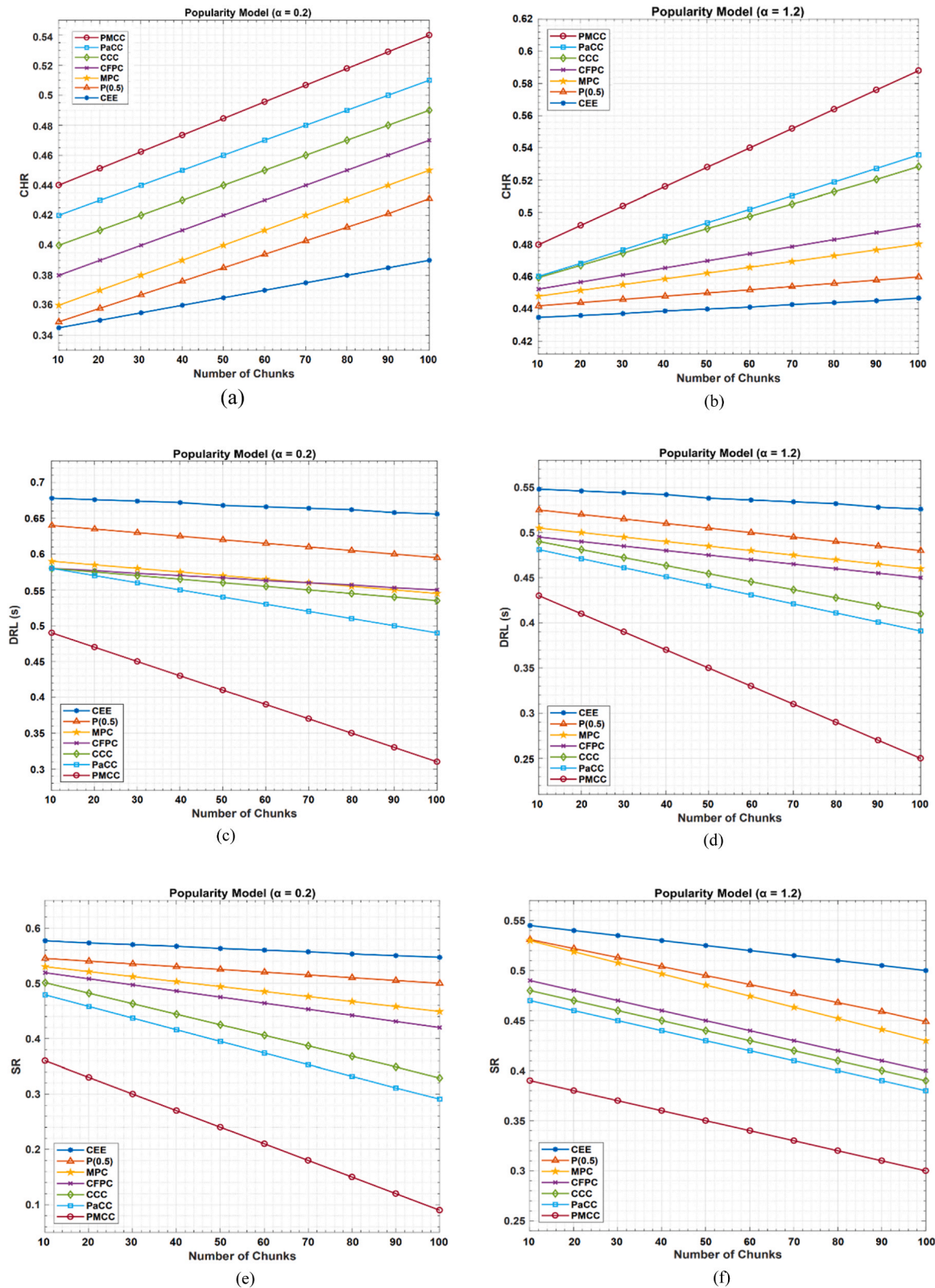


Fig. 3. Simulation results on CHR, DRL, and SR with Constant α value.

contents need to be fetched from distant data providers, increasing the data retrieval latency and SR, and DRL. PaCC introduced a metric PAC to determine the content caching node, where contents are cached at the closeness centrality nodes. It caches the content based on PAC value. Nevertheless, the number of contents is much greater than the available

cache storage. Therefore, not all incoming contents can be cached at one node. As a result, content eviction increases, and most of the users' Interests are forwarded to the server to download the demanded contents. This reduction in caching performance is evident in terms of SR, latency, and hit rate. From Fig. 2, it is clear that the proposed PMCC strategy

Table 4

Performance Metrics and Performance of the caching strategies.

Metrics	Performance	PMCC	CaCC	CCC	CFPC	MPC	P (0.50)	CEE
Cache Hit Ratio	Absolute	✓	X	X	X	X	X	X
	High	X	✓	X	X	X	X	X
	Better	X	X	✓	X	X	X	X
	Moderate	X	X	X	✓	✓	X	X
	Low	X	X	X	X	X	✓	X
Data Retrieval Delay	Poor	X	X	X	X	X	X	✓
	Absolute	✓	X	X	X	X	X	X
	High	X	✓	X	X	X	X	X
	Better	X	X	✓	X	X	X	X
	Moderate	X	X	X	✓	X	X	X
Stretch Ratio	Low	X	X	X	X	✓	✓	X
	Poor	X	X	X	X	X	X	✓
	Absolute	✓	X	X	X	X	X	X
	High	X	✓	X	X	X	X	X
	Better	X	X	✓	✓	X	X	X
CEE	Moderate	X	X	X	X	X	X	X
	Low	X	X	X	X	X	✓	X
	Poor	X	X	X	X	X	X	✓
	Absolute	✓	X	X	X	X	X	X
	High	X	✓	X	X	X	X	X

outperformed than CEE, ProbCache, MPC, CFPC, CCC, and CaCC using all selected metrics. From the experiment results, the PMCC has performed better than other strategies and it can perform better in terms of reducing the cache SR because PMCC does not cache only highly downloaded data contents and also selects the less popular data contents according to the required demand and cache them at appropriate network nodes close the end users. According to PMCC, the diverse data contents are selected based on their downloading frequencies, and multiple data contents are selected instead of one data item and cache the selected data contents at highly requested locations. Therefore, there is very little chance in PMCC to cache redundant popular data contents again and again at the location it does not need. Fig. 3 illustrates the simulation results on the constant popularity values and varying cache sizes. Fig. 3(a, b) shows the simulation results on the cache hit ratio, (c, d) represents the outcomes of DRL, and (e, f) illustrates the performance of different caching strategies on SR. Fig. 3 shows the different results as compared to the constant cache size. The reason is that with varying cache sizes and the performance increases with the increasing cache size because the transmitted data content gets more accommodation to cache. Therefore, the performance is increased. Similarly, the data retrieval latency and average hop count show similar performance to the cache hit ratio because of additional caching storage. As compared to the varying popularity parameter the strategies achieve better performance with varying cache sizes. The reason is that more cache size shows that the additional data contents can be cached near the end users and therefore, the performance is improved. In this case, the cache size is taken as varying from 10 chunks to 100 chunks while the weight factor (α) is chosen as constant, and its values are selected as 0.2 and 1.2.

Fig. 3(a, b, c, d, e, and f) show the caching performance on cache hit, latency and stretch using fixed (α) values. From the given figures, it is clear that all the caching strategies achieve better average ST performance with increasing cache sizes. The reason is that there is a chance to accommodate additional contents at the caching nodes and therefore, most of the users can fetch their contents from the nearest caching nodes. The CEE performs poorly with all cache sizes regarding the ProbCache, MPC, CFPC, CCC, PaCC, and PMCC. According to Fig. 3, it achieved the lowest result with 10 chunks cache size, and with 100 chunks, it improves the performance but still, it is poor than other comparing strategies. Moreover, ProbCache shows similar performance because of its caching algorithms. Both strategies perform better than poor to achieve higher results in terms of selected metrics. Moreover, CFPC improves performance in terms of all metrics than CEE and ProbCache. However, the PMCC bests the CEE, ProbCache, MPC, CCC, PaCC, and CFPC in terms of achieving better performance with all the caching sizes in terms of all performance metrics as shown in Fig. 3. Table 4 shows the accumulation performance of the comparing caching

strategies.

7. Conclusion and future directions

In this work, a novel caching approach is introduced for the selection of caching content in the IoT context that considers ICN and is known as PMCC, which uses multiclass content categorized into HDLC and LDLC by the number of downloads. The chosen contents are stored at nodes with the highest levels of request generation, thus guaranteeing that frequently used data is available in places with high demands for it. The ndnSim simulator was used to evaluate the performance of PMCC, as this library is considered the most popular for simulating NDN contexts. In this paper, PMCC is tested against various caching methods such as CEE, P(0.5), MPC, CFPC, CCC, and PaCC. The evaluation metrics were selected as cache hit ratio, latency, and stretch ratio. The outcomes of the current investigation show that the proposed PMCC outperforms all these existing caching strategies, substantiating the practical efficiency of the identified method.

In the future, PMCC would be extended to include evaluations of energy and bandwidth for the validation of efficiency. What make the PMCC strategy applicable in different environments are the prospects of the field and application of fog and edge computing based on ICN and NDN. Besides, it can be used to optimize data dissemination procedures in emerging wireless networks such as 5 G and 6 G. Besides these, PMCC can be incorporated into Mobile Ad-hoc Networks (MANET), Vehicular Ad-hoc Networks (VANET), and Software Defined Networking (SDN). The integration of such caching would improve the performance of the caching networks, something that makes PMCC a future-proof solution for networks of the future. Alleviating today's issues and adopting the latest technological developments, PMCC forms the basis for improving data caching management in multiple IoT contexts.

CRedit authorship contribution statement

Meng Yahui: Methodology, Formal analysis. **Ahmad Abrar:** Supervision. **Muhammad Ali Naeem:** Writing – review & editing, Writing – original draft, Validation, Methodology, Formal analysis, Conceptualization.

Declaration of Competing Interest

Both Authors do not have any conflict of interest.

Acknowledgements

This research is supported by the Projects of Talents Recruitment of

Guangdong University of Petrochemical Technology (GDUPT) (NO. 2022rcyj2015) in Guangdong Province, People's Republic of China.

References

- [1] J.S. Yalli, M.H. Hasan, and A. Badawi, "Internet Of Things (IoT): Origin, Embedded Technologies, Smart Applications and its Growth in the Last Decade," *IEEE Access*, 2024.
- [2] Y. Meng, A.B. Ahmad, M.A. Naeem, Comparative Analysis of Popularity aware Caching Strategies in Wireless-based NDN based IoT Environment, *IEEE Access* (2024).
- [3] A. Rajagopalan, D. Swaminathan, M. Bajaj, I. Damaj, R.S. Rathore, A.R. Singh, V. Blazek, L. Prokop, "Empowering power distribution: Unleashing the synergy of IoT and cloud computing for sustainable and efficient energy systems, *Results Eng.* (2024) 101949.
- [4] Y. Meng, A.B. Ahmad, Performance measurement through caching in named data networking based internet of things, *IEEE Access* (2023).
- [5] A. Anjum, P. Agbaje, A. Mitra, E. Oseghale, E. Nwafor, H. Olufowobi, Towards named data networking technology: Emerging applications, use cases, and challenges for secure data communication, *Future Gener. Comput. Syst.* 151 (2024) 12–31.
- [6] Y. Meng, M.A. Naeem, M. Sohail, A.K. Bashir, R. Ali, and Y. Bin Zikria, "Elastic caching solutions for content dissemination services of ip-based internet technologies prospective (Sep, 10.1007/s11042-020-09990-4, 2020), *Multimed. Tools Appl.* 80 (2021) 17023.
- [7] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey, *IEEE Internet Things J.* 10 (2023) 14595–14620.
- [8] M.A. Naeem, R. Ali, M. Alazab, M. Yhui, Y.B. Zikria, "Enabling the content dissemination through caching in the state-of-the-art sustainable information and communication technologies, *Sustain. Cities Soc.* (2020) 102291.
- [9] D. Doan Van, Q. Ai, In-network caching in information-centric networks for different applications: a survey, *Cogent Eng.* 10 (2023) 2210000.
- [10] S. Mishra, V.K. Jain, K. Gyoda, S. Jain, Distance-based dynamic caching and replacement strategy in NDN-IoT networks, *Internet Things* (2024) 101264.
- [11] A. Javaheri, A. Bohlooli, K. Jamshidi, Enhancing computation reuse efficiency in ICN-based edge computing by modifying content store table structure, *Computing* (2024) 1–21.
- [12] H. Zhu, Y. Cao, X. Wei, W. Wang, T. Jiang, S. Jin, "Caching transient data for Internet of Things: a deep reinforcement learning approach, *IEEE Internet Things J.* 6 (2018) 2074–2083.
- [13] L. Leira, M. Luís, S. Sargento, Context-based caching in mobile information-centric networks, *Comput. Commun.* 193 (2022) 214–223.
- [14] G. Xylomenos, C.N. Ververidis, V.A. Siris, N. Fotiou, C. Tsilopoulos, X. Vasilakos, K. V. Katsaros, G.C. Polyzos, "A survey of information-centric networking research," *IEEE Commun. Surv. Tutor.* 16 (2013) 1024–1049.
- [15] M. Meddeb, A. Dhraief, A. Belghith, T. Monteil, and K. Drira, "Named Data Networking: A promising architecture for the Internet of things (IoT)," 2017.
- [16] S. Arshad, M.A. Azam, M.H. Rehmani, J. Loo, "Recent advances in information-centric networking-based Internet of Things (ICN-IoT), *IEEE Internet Things J.* 6 (2018) 2128–2158.
- [17] H. Yan, D. Gao, W. Su, C.H. Foh, H. Zhang, A.V. Vasilakos, "Caching strategy based on hierarchical cluster for named data networking," *IEEE Access* 5 (2017) 8433–8443.
- [18] B. Nour, K. Sharif, F. Li, H. Mounsla, A.E. Kamal, H. Affi, NCP: A near ICN cache placement scheme for IoT-based traffic class, 2018 *IEEE Glob. Commun. Conf. (GLOBECOM)* (2018) 1–6.
- [19] J. Pfender, A. Valera, W.K. Seah, Performance comparison of caching strategies for information-centric IoT, *Proc. 5th ACM Conf. Inf. -Centr Netw.* (2018) 43–53.
- [20] M. Meddeb, A. Dhraief, A. Belghith, T. Monteil, K. Drira, H. Mathkour, Least fresh first cache replacement policy for NDN-based IoT networks, *Pervasive Mob. Comput.* 52 (2019) 60–70.
- [21] A. Aboodi, T.-C. Wan, G.-C. Sodhy, "Survey on the Incorporation of NDN/CCN in IoT," *IEEE Access* 7 (2019) 71827–71858.
- [22] J. Pfender, A. Valera, W.K. Seah, Easy as ABC: A lightweight centrality-based caching strategy for information-centric IoT, *Proc. 6th ACM Conf. Inf. -Centr Netw.* (2019) 100–111.
- [23] M. Amadeo, G. Ruggeri, C. Campolo, A. Molinaro, G. Mangiullo, Caching popular and fresh IoT contents at the edge via named data networking, *IEEE INFOCOM 2020-IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)* (2020) 610–615.
- [24] H. Asmat, F. Ullah, M. Zareei, A. Khan, E.M. Mohamed, "Energy-efficient centrally controlled caching contents for information-centric internet of things," *IEEE Access* 8 (2020) 126358–126369.
- [25] O. Serhane, K. Yahyaoui, B. Nour, H. Mounsla, Energy-aware cache placement scheme for IoT-based ICN networks, *ICC 2021-IEEE Int. Conf. Commun.* (2021) 1–6.
- [26] B. Alahmri, S. Al-Ahmadi, A. Belghith, "Efficient pooling and collaborative cache management for NDN/IoT networks," *IEEE Access* 9 (2021) 43228–43240.
- [27] M.A. Naeem, T.N. Nguyen, R. Ali, K. Cengiz, Y. Meng, T. Khurshaid, Hybrid cache management in IoT-based named data networking, *IEEE Internet Things J.* 9 (2021) 7140–7150.
- [28] G. Dhawan, A.P. Mazumdar, Y.K. Meena, CNCP: A Candidate Node Selection for Cache Placement in ICN-IoT, 2022 *IEEE 6th Conf. Inf. Commun. Technol. (CICT)* (2022) 1–6.
- [29] N. Dinh, Y. Kim, An Energy Reward-Based Caching Mechanism for Information-Centric Internet of Things, *Sensors* 22 (2022) 743.
- [30] M.A. Naeem, R. Ullah, Y. Meng, R. Ali, B.A. Lodhi, Caching Content on the network layer: a performance analysis of caching schemes in ICN-based internet of things, *IEEE Internet Things J.* (2021).
- [31] M.A. Naeem, T.N. Nguyen, R. Ali, K. Cengiz, Y. Meng, T. Khurshaid, Hybrid cache management in IoT-based named data networking, *IEEE Internet Things J.* (2021).
- [32] O.A. Arqub, Z. Abo-Hammour, Numerical solution of systems of second-order boundary value problems using continuous genetic algorithm, *Inf. Sci.* 279 (2014) 396–415.
- [33] Z.S. Abo-Hammour, M.I. Al Saaideh, M. Alkayyali, H.J. Khasawneh, Optimal design of lead compensator using nature-inspired algorithms, 2019 *IEEE Jordan Int. Jt. Conf. Electr. Eng. Inf. Technol. (JEEIT)* (2019) 40–45.
- [34] Z. Abo-Hammour, O.A. Arqub, O. Alsmadi, S. Momani, A. Alsaedi, "An optimization algorithm for solving systems of singular boundary value problems," *Appl. Math. Inf. Sci.* 8 (2014) 2809.
- [35] Y. Liu, T. Zhi, H. Xi, X. Duan, H. Zhang, A novel content popularity prediction algorithm based on auto regressive model in information-centric IoT, *IEEE Access* 7 (2019) 27555–27564.
- [36] M.A. Naeem, S.A. Nor, S. Hassan, B.-S. Kim, "Performances of probabilistic caching strategies in content centric networking," *IEEE Access* 6 (2018) 58807–58825.
- [37] M.A. Naeem, M.A.U. Rehman, R. Ullah, B.-S. Kim, "A comparative performance analysis of popularity-based caching strategies in named data networking," *IEEE Access* 8 (2020) 50057–50077.
- [38] M.A. Naeem, S.A. Nor, S. Hassan, B.-S. Kim, Performances of probabilistic caching strategies in content centric networking, *IEEE Access* (2018).
- [39] M. Zhang, B. Hao, R. Wang, Y. Wang, "A pre-caching strategy based on the content relevance of smart device's request in information-centric IoT," *IEEE Access* 8 (2020) 75761–75771.
- [40] Q. Chen, R. Xie, F.R. Yu, J. Liu, T. Huang, Y. Liu, "Transport control strategies in named data networking: a survey," *IEEE Commun. Surv. Tutor.* 18 (2016) 2052–2083.
- [41] M. Zhang, H. Luo, H. Zhang, A survey of caching mechanisms in information-centric networking, *IEEE Commun. Surv. Tutor.* 17 (2015) 1473–1499.
- [42] M. Amadeo, C. Campolo, G. Ruggeri, A. Molinaro, Popularity-aware closeness based caching in NDN edge networks, *Sensors* 22 (2022) 3460.